- This algorithm doesn't find an optimal solution for all problems.
- An optimal solution is one that maximizes/minimizes one OR both of these factors: Time and/or Cost
- Greed factor is a value on basis of which you make the local optimal decision.

An optimization problem can be expressed as:

Optimize $f(x)$ *subject to* $x \in S$

where:

- $f(x)$= **objective function** (the thing to maximize or minimize)
- x= set of **decision variables** → Coins Available
- S= **feasible region** (set of all possible values of x that satisfy the constraints)

↳ coins sum to 18

Principle of optimality:

An optimization that is done through a sequence of decisions

General Steps

1. **Initialize Empty Solution Set**
   Begin with no elements in the solution.
2. **Select Best Available Choice**
   Choose the most promising option at each step.
3. **Check Feasibility**
   Ensure the current solution remains valid.

4 **Update Solution**

Add the chosen element if feasible, otherwise discard.

5 **Repeat Until Solution Found**

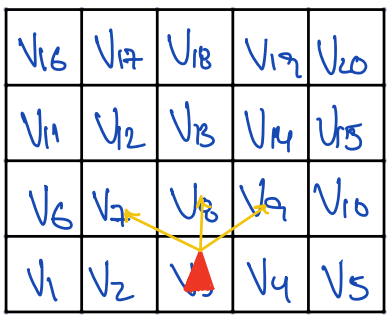
Continue the process until the problem is solved.

NOTE: six likes board-type problems

Example problem done in class

Let's say you have a grid, where a player can appear at any start point. Each location/cell has a value.

The player can only move forward or diagonally. The goal is maximize the accumulated value to destination.



Another variant would have a fixed starting point. Such problems are called multi-stage graph problems & they can't be solved by naive greedy approach.

Example: Coin Change Problem

1. **Problem Statement**

   Make change for $18 using the fewest coins possible.

2. **Available Coins**

   $5, $2, and $1 coins, unlimited quantity of each.

3. **Greedy Approach**

   Always choose the largest coin that doesn't exceed the remaining amount.

```
sol      = [ ]
coins = [5, 2, 1]
rem_amt = 18

while rem_amt > 0:

    if  rem_amt - coins[0] ≥ 0
        sol.add(5)

    else if rem_amt - coins[1] ≥ 0
        sol.add(2)

    else if rem_amt - coins[2] ≥ 0
        sol.add(1)
```

Dry Run:

18 − 5 = 13 ≥ 0
sol = [5]

13 − 5 = 8 ≥ 0
sol = [5, 5]

8 − 5 = 3 ≥ 0
sol = [5, 5, 5]

8 − 2 = 1 ≥ 0
sol = [5, 5, 5, 2]

1 − 1 = 0 ≥ 0
Ans ← sol = [5, 5, 5, 2, 1]

2. 📅 Activity Selection Problem (Maximum Events)

Problem: Given `n` activities with start and end times, select the **maximum number of non-overlapping activities.**

Example:

Activity	Start	End
A1	1	4
A2	3	5
A3	0	6
A4	5	7

A5	3	9
A6	6	10
A7	8	11

```
sol = []
times = [(1,4), (3,5), ..., (8,11)]
prevEnd = 0

for start, end in sorted(times, key = lambda x: x[1]):
    if start > prevEnd:
        sol.add((start, end))
        prevEnd = end
```

Dry Run:

1 > 0
sol = [(1,4)]
prevEnd = 4

3 > 4
0 > 4
5 > 4
sol = [(1,4), (5,7)]
prevEnd = 7

3 > 7
6 > 7
8 > 7
sol = [(1,4), (5,7), (8,11)] → Ans
prevEnd = 11

Knapsack Problem

- A set of items, each with a weight (w_i) & value (v_i)
- A maximum capacity W
- Objective is to maximize $\sum v_i$ such that $\sum w_i \leq W$

Fractional knapsack: Fraction of an item can be taken

0/1 knapsack: An item is either left out OR taken (no fractions allowed)

Fractional knapsack Example

```
values= [60, 100, 120]
weights=[10, 20, 30]
v_ws= values/weights
W = 50
sol= 0

for v_w, v, w in sort(zip(v_ws, values, weights), ascending=False)
    if W-w >=0:
        sol+= v
    else:
        sol+= v*(W/w)
```

Dry Runs

$50 - 10 = 40 \geq 0$

sol = 60

$40 - 20 = 20 \geq 0$

sol = 160

$20 - 30 = -10 \geq 0$

else

$sol = 160 + 120 * \left(\frac{20}{30}\right)$

sol = 240 → Ans

0/1 knapsack Example

- Greedy approach can't solve 0/1 knapsack problem

Huffman Encoding

- message "AAAABBBCCCDDEEF" is sent as ASCII b/w sender & reciever.
- Each ASCII character can be represented with 8 bits
- Total message length without compression:

  $15 \times 8 = 120$ bits

- There are two ways to compress this:

1. Fixed length encoding

Char	Frequency	Code
A	4	000
B	3	001
C	3	010
D	2	011
E	2	100
F	1	101

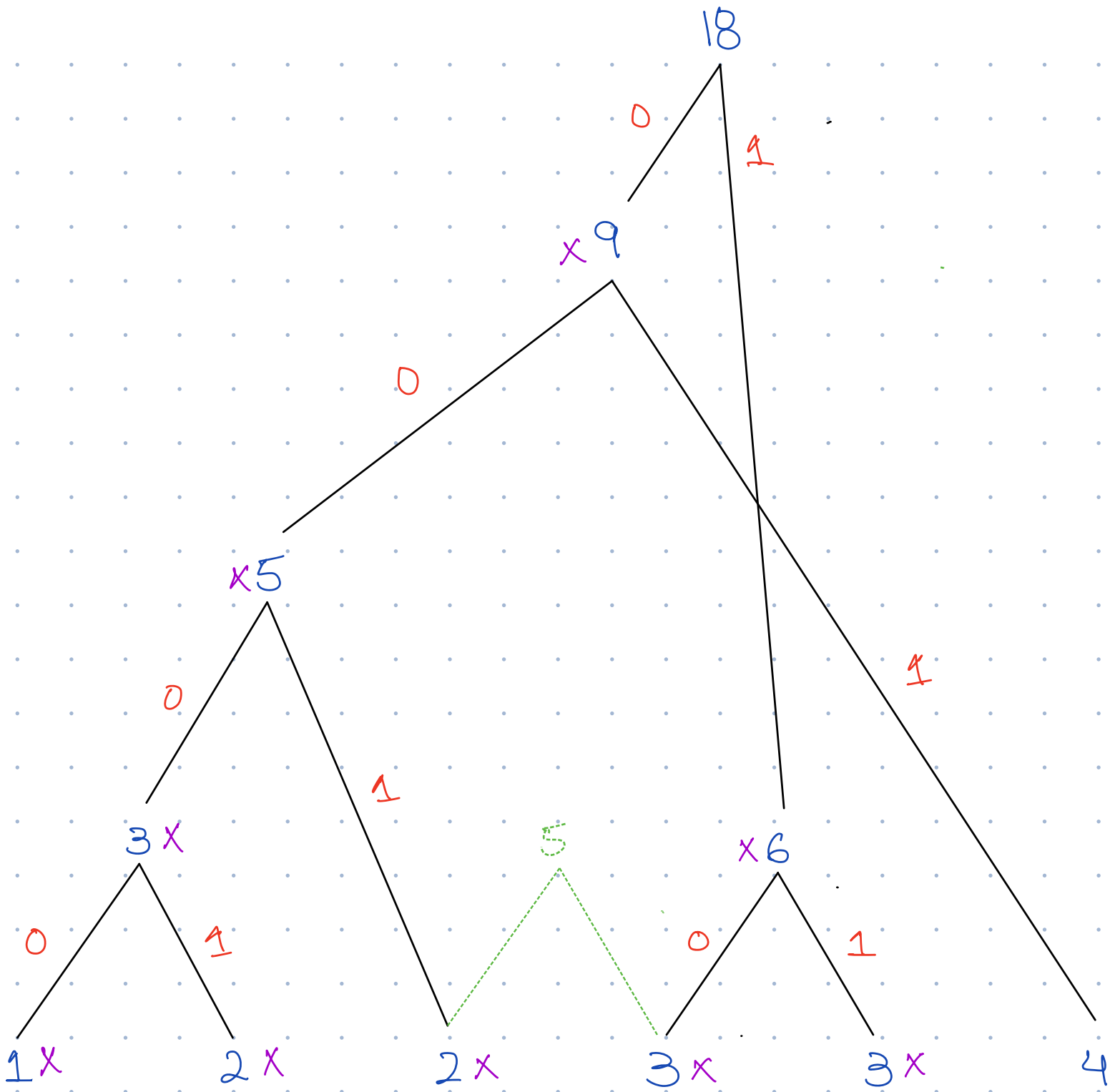
→ As there 6 chars, we need atleast 3 bits

Total message length: $15 \times 3 + [6 \times 8 + 6 \times 3]$

- 15×3: new message size = 111 bits
- 6×8: 6 ASCII chars
- 6×3: 3bit codes for each ASCII char
- [ ]: Encoding table we need to send this aswell to allow the recieved decode the message.

2. Variable length (Huffman) encoding

1. Count frequency of each character
2. Insert all characters into a min-heap
3. Extract two minimum-frequency nodes
4. Create a new internal node = sum of those frequencies
5. Insert the new node back into the heap
6. Repeat until one node remains
7. Assign codes: left = 0, right = 1



Char	Code	Frequency
A	01	4
B	11	3
C	10	3
D	001	2
E	0001	2
F	0000	1

$O(n \log n)$

Total message length:

$$\Big[(4\times2)+(3\times2)+(3\times2)+(2\times3)+(2\times4)+(1\times4)\Big] \text{ new message size}$$

$$+$$

$$\Big[6\times8+\{2+2+2+3+4+4\}\Big] \text{ code table size}$$

103 bits

Correctness check: No code can be prefix of another (i.e if 'A' is '01', B can't be '010')
↳ can't be prefix

Job Sequencing:

- You have n Jobs, each with a deadline & profit.
- Each Job takes 1 unit of time.
- Our objective to maximize total profit.
- Our constraint is that selected jobs must be completed before its deadline.
- Greedy Factor: Choose the job with the highest profit.
  → This will not give optimal solution

Now, we can perform the selected job now OR do it just before its deadline.
↳ This will give optimal solution

Example,

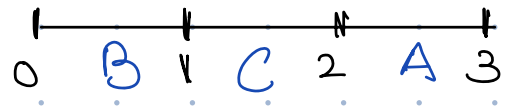
Jobs	Deadline	Profit
A	3	100
B	1	100
C	2	99
D	3	98

$O(n^2)$

option 01:

profit = 100 + 99 + 98

option 02:



profit = 100 + 100 + 99

Input: Profits[], Deadlines[]

Steps:

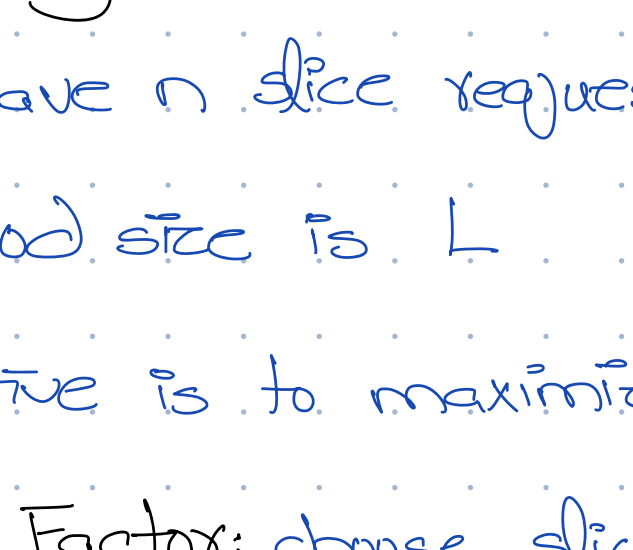
1. Sort jobs by decreasing profit
2. Find max deadline `maxD`
3. Create an array `slots[maxD]` initialized as empty
4. For each job in sorted order:
    1. For slot = deadline → 1:
        1. If slot is empty → assign job
        2. Break
5. Output scheduled jobs + total profit

Rod Cutting Problem

- You have n slice requests, each with a value & length.
- Your rod size is L
- Objective is to maximize total value
- Greed Factor: choose slice with max value/length

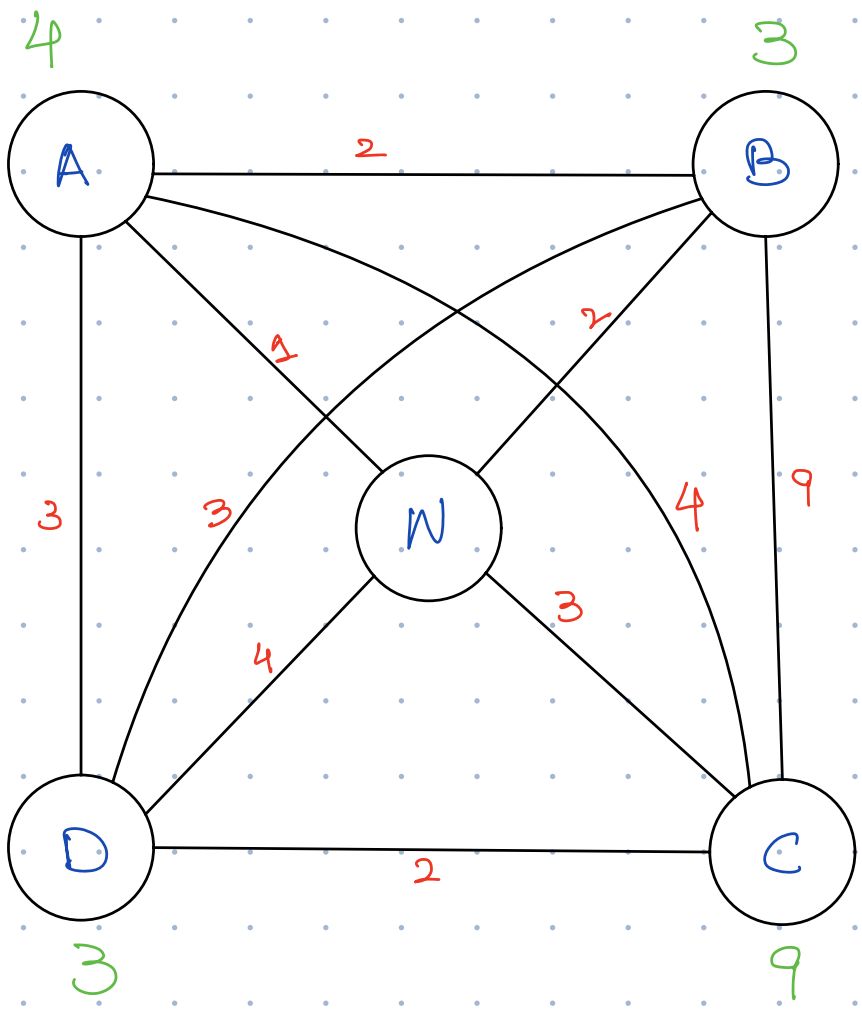
Example,

Slice	length	value	V/L	Length = 4
A	3	19	6.3	
B	4	20	5	



NOTE: Greedy can't solve this.

Routing Algorithms:



- Two vehicles with max capacity = 10
- Always start & stop at W.

Goal: Minimize total distance travelled.

V_1: W $\xrightarrow{1}$ A $\xrightarrow{2}$ B $\xrightarrow{3}$ D $\xrightarrow{4}$ W
(10, 6, 3, 0)

V_2: W $\xrightarrow{3}$ C $\xrightarrow{3}$ W
(10, 7)

Greed factor: Travel to nearest unvisited node given you have capacity remaining for it.

Total distance = (1+2+3+4) + (3+3)

= 16

NOTE: Although this is a single source multi-stage problem, it is being solved by greedy. Thus, if multiple factors are involved such as cost & capacity in above example then greedy MAY solve them.